

Chapter 11: Memory Allocation

In this chapter, we'll meet `malloc`, C's dynamic memory allocation function, and we'll cover dynamic memory allocation in some detail.

As we begin doing dynamic memory allocation, we'll begin to see (if we haven't seen it already) what pointers can really be good for. Many of the pointer examples in the previous chapter (those which used pointers to access arrays) didn't do all that much for us that we couldn't have done using arrays. However, when we begin doing dynamic memory allocation, pointers are the only way to go, because what `malloc` returns is a pointer to the memory it gives us. (Due to the equivalence between pointers and arrays, though, we will still be able to think of dynamically allocated regions of storage as if they were arrays, and even to use array-like subscripting notation on them.)

You have to be careful with dynamic memory allocation. `malloc` operates at a pretty "low level"; you will often find yourself having to do a certain amount of work to manage the memory it gives you. If you don't keep accurate track of the memory which `malloc` has given you, and the pointers of yours which point to it, it's all too easy to accidentally use a pointer which points "nowhere", with generally unpleasant results. (The basic problem is that if you assign a value to the location pointed to by a pointer:

```
*p = 0;
```

and if the pointer `p` points "nowhere", well actually it can be construed to point somewhere, just not where you wanted it to, and that "somewhere" is where the 0 gets written. If the "somewhere" is memory which is in use by some other part of your program, or even worse, if the operating system has not protected itself from you and "somewhere" is in fact in use by the operating system, things could get ugly.)

[11.1 Allocating Memory with `malloc`](#)

[11.2 Freeing Memory](#)

[11.3 Reallocating Memory Blocks](#)

[11.4 Pointer Safety](#)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)